- ***3*** Creates the support tensor, a tensor of evenly spaced values from –10 to +10
- ***4*** Plots the distribution as a bar plot

We have defined a uniform probability distribution; now let's see how we update the distribution. We want a function, `update_dist(z, reward)`, that takes a prior distribution and an observed reward and returns a posterior distribution. We represent the support of the distribution as a vector from –10 to 10:

```
>>> support
array([-10. ,  -9.6,  -9.2,  -8.8,  -8.4,  -8. ,  -7.6,  -7.2,
-6.8,
        -6.4,  -6. ,  -5.6,  -5.2,  -4.8,  -4.4,  -4. ,  -3.6,
-3.2,
        -2.8,  -2.4,  -2. ,  -1.6,  -1.2,  -0.8,  -0.4,   0. ,
0.4,
         0.8,   1.2,   1.6,   2. ,   2.4,   2.8,   3.2,   3.6,   4.
,
         4.4,   4.8,   5.2,   5.6,   6. ,   6.4,   6.8,   7.2,
7.6,
         8. ,   8.4,   8.8,   9.2,   9.6,  10. ])
```

We need to be able to find the closest support element in the support vector to an observed reward. For example, if we observe $r_t = –1$, we'll want to map that to either –1.2 or –0.8 since those are the closest (equally close) support elements. More importantly, we want the indices of these support elements so that we can get their corresponding probabilities in the probability vector. The support vector is static—we never update it. We only update the corresponding probabilities.

You can see that each support element is 0.4 away from its nearest neighbors. The numpy `linspace` function creates a sequence of evenly spaced elements, and the spacing is given by $\frac{v_{max} - v_{min}}{N - 1}$, where N is the number of support elements. If you plug 10, –10, and $N = 51$ into that formula, you get 0.4. We call this value *dz* (for delta Z), and we use it to find the closest support element index value by the equation $b_j = \frac{r - v_{min}}{dz}$, where b_j is the index value. Since b_j may be a fractional number, and